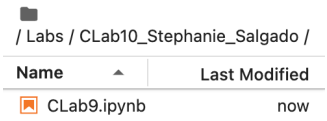
CLab 10

Stephanie Salgado

Setup



I'll be using the same "CLab9.ipynb" file from the last lab.

Submitting the slice

```
slice.submit()
```

```
Retry: 11, Time: 277 sec
```

Slice

ID	3bf22111-37f7-4488-801a-0dc4100dfa57
Name	Stephanie_Salgado_CLab10
Lease Expiration (UTC)	2024-11-19 18:40:59 +0000
Lease Start (UTC)	2024-11-18 18:40:59 +0000
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
State	StableOK

Since I covered most of this for the last lab, I'll skip over including everything. I run the same cells from the same file and after that, I submitted the slice.

Logging into resources

```
Last login: Mon Nov 18 18:42:53 2024 from 2610:1e0:1700:205::51
ubuntu@romeo:~$
```

```
Last login: Mon Nov 18 18:42:58 2024 from 2610:1e0:1700:205::51
ubuntu@router:~$
```

```
Last login: Mon Nov 18 18:42:53 2024 from 2610:1e0:1700:205::51
ubuntu@server:~$
```

Once again, I run the cells to configure the resources and when I finish, I log into each node using the SSH commands.

Web access

Installing lynx to browse the web from the terminal

```
ubuntu@romeo:~$ sudo apt -y install lynx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libidn11 lynx-common
The following NEW packages will be installed:
  libidn11 lynx lynx-common
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 1586 kB of archives.
After this operation, 5731 kB of additional disk space will be used.
Get:1 http://nova.clouds.archive.ubuntu.com/ubuntu focal/main amd64 libidn11 amd64 1.33-2.2ubuntu2 [46.2 kB]
Get:2 http://nova.clouds.archive.ubuntu.com/ubuntu focal/universe amd64 lynx-common all 2.9.0dev.5-1 [914 kB]
Get:3 http://nova.clouds.archive.ubuntu.com/ubuntu focal/universe amd64 lynx amd64 2.9.0dev.5-1 [626 kB]
Fetched 1586 kB in 1s (1497 kB/s)
Selecting previously unselected package libidn11:amd64.
(Reading database ... 95235 files and directories currently installed.)
Preparing to unpack .../libidn11_1.33-2.2ubuntu2_amd64.deb ...
Unpacking libidn11:amd64 (1.33-2.2ubuntu2) ...
Selecting previously unselected package lynx-common.
Preparing to unpack .../lynx-common_2.9.0dev.5-1_all.deb ...
Unpacking lynx-common (2.9.0dev.5-1) ...
Selecting previously unselected package lynx.
```

On “romeo”, I use “sudo apt update” to make sure everything is up to date. Then, I use “sudo apt -y install lynx” to install the lynx package.

Installing the Apache web server

```
ubuntu@server:~$ sudo apt -y install apache2
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

On “server”, I use “sudo apt -y install apache2” to install Apache web server.

Generating a self-signed certificate and key

```
ubuntu@server:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/ssl-cert-snakeoil.key -out /etc/ssl/certs/ssl-cert-snakeoil.pem
Generating a RSA private key
.....................................+++++
.........+++++
writing new private key to '/etc/ssl/private/ssl-cert-snakeoil.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:Texas
Locality Name (eg, city) []:San Antonio
Organization Name (eg, company) [Internet Widgits Pty Ltd]:TAMUSA
Organizational Unit Name (eg, section) []:CSCI
Common Name (e.g. server FQDN or YOUR name) []:server
```

Still on “server”, I use “sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/ssl-cert-snakeoil.key -out /etc/ssl/certs/ssl-cert-snakeoil.pem”.

Editing the config file for the SSL-enabled version

```
  GNU nano 4.8                                  /etc/apache2/sites-available/default-ssl.conf
<IfModule mod_ssl.c>
        <VirtualHost _default_:443>
                ServerName server
                ServerAdmin webmaster@localhost

                DocumentRoot /var/www/html

                # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
                # error, crit, alert, emerg.
```

I use “sudo nano /etc/apache2/sites-available/default-ssl.conf”, and add “ServerName server” under “<VirtualHost _default_:433>”.

Enabling the SSL module for Apache

```
ubuntu@server:~$ sudo a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
```

I used “sudo a2enmod ssl”.

Enabling the new SSL-enabled site

```
ubuntu@server:~$ sudo a2ensite default-ssl.conf
Enabling site default-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

After that, I used “sudo a2ensite default-ssl.conf”.

Restarting the service

```
ubuntu@server:~$ sudo systemctl restart apache2
```

Lastly, I used “sudo systemctl restart apache2”.

Creating the “form” HTML file

```
 GNU nano 4.8
<!DOCTYPE html>
<html>
  <body>
    <form action="/done.html">
      <label for="fname">First name:</label><br>
      <input type="text" id="fname" name="fname" value="John"><br>
      <label for="lname">Last name:</label><br>
      <input type="text" id="lname" name="lname" value="Doe"><br><br>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

I used “sudo nano /var/www/html/form.html”, then added what’s above.

Creating the “done” HTML file

```
  GNU nano 4.8
<!DOCTYPE html>
<html>
  <body>
    <h1>Done!</h1>
  </body>
</html>
```

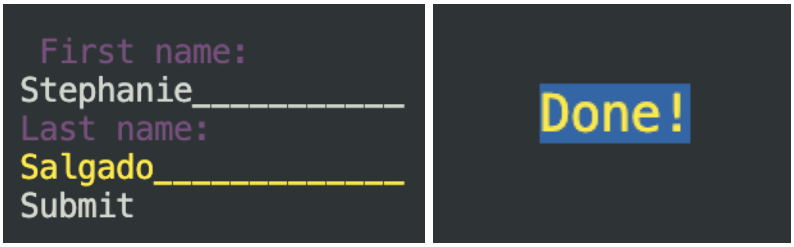
Similarly, I use “sudo nano /var/www/html/done.html”.

Capturing traffic (HTTP)

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.10.2.100 | grep -oP "(?<=dev )[^ ]+") -w security-http-$(hostname -s).pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

On “romeo”, I capture, then store the traffic in the “security-http-…“ file.

Initiating a HTTP session




```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.10.2.100 | grep -oP "(?<=dev )[^ ]+") -w security-http-$(hostname -s).pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C28 packets captured
28 packets received by filter
0 packets dropped by kernel
```

On another “romeo” terminal, I use “lynx http://server/form.html”. Then, I enter my first and last name in the form fields. I exit then stop the capture.

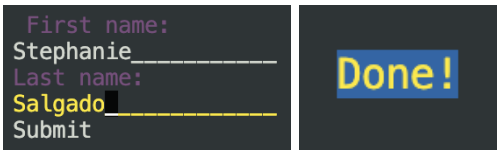
Capturing traffic (HTTPS)

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.10.2.100 | grep -oP "(?<=dev )[^ ]+") -w security-https-$(hostname -s).pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

This time, I capture and store the traffic in the “security-https-…” file.

Initiating a HTTPS session

```
SSL error:The certificate is NOT trusted. The certificate issuer is unknown. -Continue? (n)
```



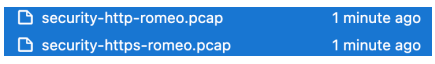

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.10.2.100 | grep -oP "(?<=dev )[^ ]+") -w security-https-$(hostname -s).pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C50 packets captured
50 packets received by filter
0 packets dropped by kernel
```

I access the form using lynx again, this time with “https”. Notice I get an “SSL error”. I fill out the form again, getting another “SSL error” after submitting.

Downloading the capture files

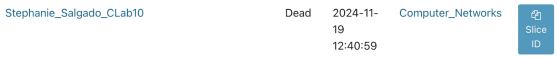
```
http_pcap = "/home/ubuntu/security-http-%s.pcap" % slice.get_node("romeo").execute("hostname", quiet=True)[0].strip()
slice.get_node("romeo").download_file("/home/fabric/work/Labs/CLab10_Stephanie_Salgado/security-http-romeo.pcap", http_pcap)
```

```
https_pcap = "/home/ubuntu/security-https-%s.pcap" % slice.get_node("romeo").execute("hostname", quiet=True)[0].strip()
slice.get_node("romeo").download_file("/home/fabric/work/Labs/CLab10_Stephanie_Salgado/security-https-romeo.pcap", https_pcap)
```



I transfer the “.pcap” files with one of the cells and download them to my Mac.

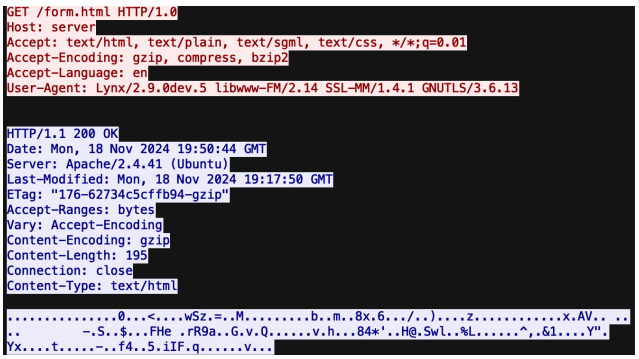
Deleting the slice

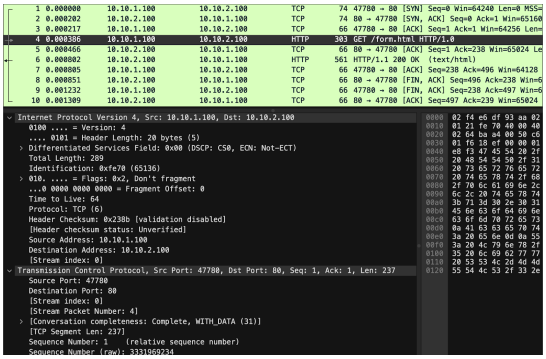


Finally, I run the cell to delete my slice and continue in Wireshark for analysis.

Exercises

1. In the packet capture of the HTTP experiment, can you read: the IP and TCP headers? The contents of the HTTP GET (including the name of the page you visited, form.html)? The data you entered in the form? Show evidence.





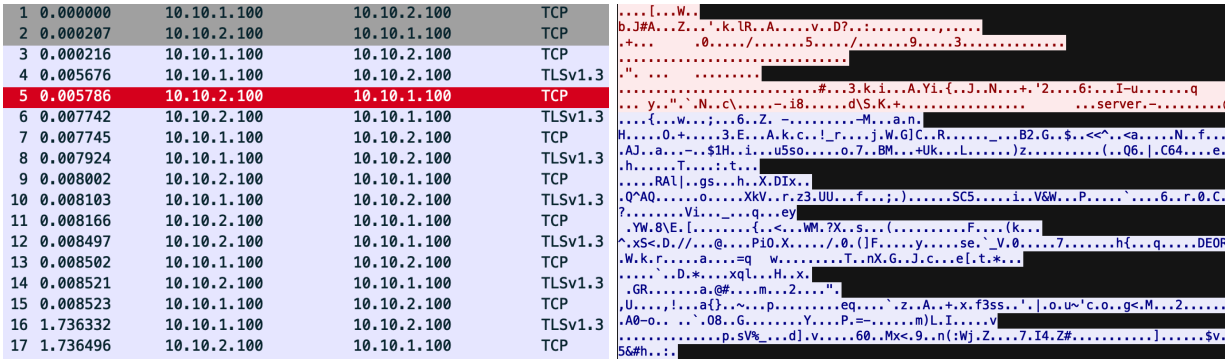
I opened the HTTP packet capture in Wireshark then selected packet 1 and used TCP Stream. Here, the contents of the “HTTP GET” are visible and include the page visited (form.html). That wasn’t even necessary since packet 4 clearly displays the same information. The IP and TCP headers are also visible when selecting different packets.

I noticed packet 18 displayed the data I entered in the form. A TCP Stream of packet 15 also shows my first and last name (which I entered in the form).

2. In the packet capture of the HTTPS experiment, can you read: the IP and TCP headers? The contents of the HTTP GET (including the name of the page you visited, form.html)? The data you entered in the form? Show evidence.

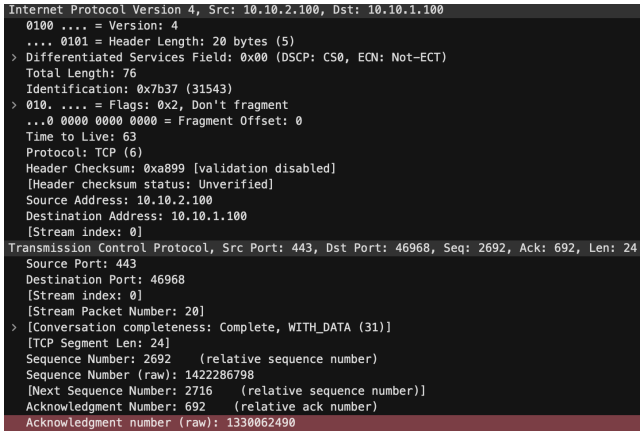
```
1  0.000000      10.10.1.100          10.10.2.100          TCP          74  46968 → 443 [SYN]
```

I can assume packet 1 is where the GET request should've been(similar to the HTTP experiment), but the name of the page visited isn't readable anywhere.



1	0.000000	10.10.1.100	10.10.2.100	TCP
2	0.000207	10.10.2.100	10.10.1.100	TCP
3	0.000216	10.10.1.100	10.10.2.100	TCP
4	0.005676	10.10.1.100	10.10.2.100	TLSv1.3
5	0.005786	10.10.2.100	10.10.1.100	TCP
6	0.007742	10.10.2.100	10.10.1.100	TLSv1.3
7	0.007745	10.10.1.100	10.10.2.100	TCP
8	0.007924	10.10.1.100	10.10.2.100	TLSv1.3
9	0.008002	10.10.2.100	10.10.1.100	TCP
10	0.008103	10.10.1.100	10.10.2.100	TLSv1.3
11	0.008166	10.10.2.100	10.10.1.100	TCP
12	0.008497	10.10.2.100	10.10.1.100	TLSv1.3
13	0.008502	10.10.1.100	10.10.2.100	TCP
14	0.008521	10.10.2.100	10.10.1.100	TLSv1.3
15	0.008523	10.10.1.100	10.10.2.100	TCP
16	1.736332	10.10.1.100	10.10.2.100	TLSv1.3
17	1.736496	10.10.2.100	10.10.1.100	TCP

After opening the HTTPS capture file, I noticed that it was longer and also that the HTTP protocol packets that showed the contents previously weren't there. I did a TCP Stream of packet 20, but noticed everything is encrypted and unreadable. I wouldn't find the data I entered in the form even if I tried.



```
Internet Protocol Version 4, Src: 10.10.2.100, Dst: 10.10.1.100
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 76
    Identification: 0x7b37 (31543)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: TCP (6)
    Header Checksum: 0xa899 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.10.2.100
    Destination Address: 10.10.1.100
    [Stream index: 0]
Transmission Control Protocol, Src Port: 443, Dst Port: 46968, Seq: 2692, Ack: 692, Len: 24
    Source Port: 443
    Destination Port: 46968
    [Stream index: 0]
    [Stream Packet Number: 20]
  > [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 24]
    Sequence Number: 2692    (relative sequence number)
    Sequence Number (raw): 1422286798
    [Next Sequence Number: 2716    (relative sequence number)]
    Acknowledgment Number: 692    (relative ack number)
    Acknowledgment number (raw): 1330062490
```

However, Wireshark still provides IP and TCP header information by default.

Summary

For this lab, I used the same “CLab9.ipynb” file to compare HTTP and HTTPS access to a simple web form. I set up the same network configuration as before, but this time, installing lynx on "romeo" and Apache web server on "server". I also generated a self-signed SSL certificate on the server. I created two HTML files, form.html and done.html, and captured network traffic for the HTTP and HTTPS sessions in which I filled out the form. While analyzing the HTTP capture, I was able to view the form data in plain text, including the data entered in the form fields. However, with HTTPS, the data was encrypted. Like with the HTTP, I could still see the IP and TCP headers. Similar to the previous lab comparing Telnet, SSH, FTP, and SFT, I was able to clearly distinguish the difference between HTTP and HTTPS.